

EngSci 721

Inverse Problems and Learning From Data

Oliver Maclaren (oliver.maclaren@auckland.ac.nz)

Module overview

Inverse Problems and Learning From Data (*Oliver Maclaren*)

[~14 lectures/3 tutorials]

1. *Basic concepts* [5 lectures + 1 Tutorial]

Forward vs inverse problems. Well-posed vs ill-posed problems. Algebra and calculus of inverse problems (left and right inverses, generalised and pseudo inverses, resolution operators, matrix calculus). Representing higher dimensional problems (image data etc).

2. *Instability and regularisation in linear and nonlinear problems* [6 lectures + 1 Tutorial]

Instability and related issues for generalised inverses. Introduction to regularisation and trade-offs. Tikhonov regularisation. Higher-order Tikhonov regularisation. Sparsity and regularisation using different norms. Truncated singular value decomposition. Iterative regularisation, including stochastic/mini-batch gradient descent.

3. *Further topics* [3 lectures + 1 Tutorial]

Regularisation parameter choice, including statistical and machine learning views of regularisation. Confidence sets for linear and nonlinear models. Physics-informed machine learning and neural networks.

Lecture 11: Stochastic/Minibatch Gradient Descent

Topics:

- Recent developments in iterative methods in machine learning: stochastic/minibatch gradient descent

Eng Sci 721 : Lecture 11.

Stochastic (minibatch) gradient descent

or

'Recent developments in iterative methods'

o gradient descent e.g. Landweber iteration

vs

Stochastic / minibatch gradient descent

Recall

Zeros & Fixed Points

[Note : here using θ for unknown]

'Theory of everything' (LeCun)

$$\boxed{F(\theta) = 0} \text{ for some } F, \theta$$

(everything is a zero of some function)

Alternative :

$$\boxed{T(\theta) = \theta} \text{ for some } T, \theta.$$

(everything is a fixed point of some function)

Relationship

if e.g.

$$\boxed{T(\theta) := \alpha F(\theta) + \theta}, \quad \alpha \neq 0$$

Then

$$\boxed{\begin{array}{c} F(\theta) = 0 \\ \Leftrightarrow \\ T(\theta) = \theta \end{array}}$$

Recall

Fixed point iterations : (define goal as fixed point of same map.)

Define:

$$\boxed{\theta_{k+1} = T(\theta_k)}$$

If $\theta_k \rightarrow \theta^*$ as $k \rightarrow \infty$

Then get $\theta^* = T(\theta^*)$ & hence
solves $F(\theta) = 0$

Contraction mappings

The above works if eg

$$\|T(\theta_1) - T(\theta_2)\| \leq a \|\theta_1 - \theta_2\| \quad \text{for all } \theta_1, \theta_2$$

& $0 < a < 1$

More detail: functional analysis
&
dynamical systems

Difficult/interesting area for eg
nonlinear T !

Recall

Normal equations for $y = X\theta$

data vector y
parameter/unknown vector θ
design matrix / forward map of data 'features' (known/measured) X

$$\bullet X^T X \theta = X^T y$$

$$\bullet F(\theta) = X^T y - X^T X \theta = 0 \quad (\text{zero eqn})$$

$$\bullet T(\theta) = \alpha F(\theta) + \theta = \theta \quad (\text{fixed point eqn})$$

Define:

$$\begin{aligned} \theta_{k+1} &= T(\theta_k) \\ &= \theta_k + \alpha F(\theta_k) \\ &= \theta_k + \alpha X^T (y - X\theta_k) \end{aligned}$$

$\rightarrow$ 'Landweber iteration'

$\rightarrow$

Note:

$$\nabla_{\theta} \|y - X\theta\|^2 = -2 X^T (y - X\theta)$$

$\Rightarrow$ gradient descent!

$$\boxed{\begin{aligned} \theta_{k+1} &= \theta_k - \alpha \nabla_{\theta} \phi(\theta_k) \\ \text{for } \phi &= \|y - X\theta\|^2, \\ \alpha &= \text{'learning rate' /} \\ &\quad \text{step size} \\ &\quad \text{param.} \end{aligned}}$$

Recall

Landweber iteration

$$\theta_{k+1} = \theta_k + \alpha X^T (y - X\theta_k)$$

&

$$0 < \alpha < 2/s_1^2$$

s_1 : largest
singular
value

Stop when eg

• $\|X\theta_k - y\| \approx \epsilon$ = measure of noise level

• 'Flattens' off (monitor)

Note: only need matrix-vector products

→ no matrix factorisations

→ efficient for large sparse systems.

Recall

Landweber iteration

Can show in linear case iterates have form

$$\theta_k = V F_k \Lambda^{-1} U^T y$$

$\underbrace{\quad}_{\text{model space basis}}$ coefficients
= Filter factors x usual SVD

$$F_k = \begin{bmatrix} f_1^k & & \\ & \ddots & \\ & & f_n^k \end{bmatrix}$$

$$f_i^k = 1 - (1 - \alpha s_i^2)^k$$

↑ using s_i for singular values

ie 'damps out singular values'
similar to Tikhonov/TSVD

Recall

Iterative regularisation: more sophisticated

→ Landweber iteration is basically
(first-order) gradient descent (for linear systems)

→ just as in optimisation, we can
choose a search/update direction
based on higher-order information

↳ more derivatives, 'larger'
local neighbourhood

General

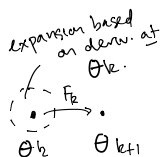
$$\theta_{k+1} = \theta_k + F_k(\theta_k, y)$$

where F_k can take into account
higher-derivatives * etc.

[* Think: Taylor series for $\theta(k+1) = T(\theta(k))$]

Trade-off: don't want it to
optimise too well!

→ want to avoid $\theta^+ = X^+ y$ limit!



Recall

Iterative regularisation:

Aster et al. (Chapter 6) Example 6.3 (see attached) → prev. lecture

→ Deblurring 2D image (200x200 pixels)

→ with & without explicit
regularisation

→ use iterative method called
'conjugate gradient least squares' (see eg 768?)

→ Blurring operator:

• 40,000 x 40,000

• very sparse

↳ SVD produces denser
matrices & requires
a lot of storage

↳ CGLS works well
& only requires
access to matrix-vector
products

→ CGLS w/out explicit regularisation
→ doesn't matter if only iterative!
→ much cheaper!

Gradient descent in machine learning?

- o Gradient descent is also widely used in machine learning (ML), particularly in training (deep) neural networks
- o While first-order optimisers (ie based on first derivatives) are not very popular in the pure optimisation community, & improved methods have also been favoured in inverse problems, first order gradient descent methods are surprisingly widely-used & competitive in ML!

↳ in particular, a family of variants usually called 'stochastic gradient descent' (SGD) is the 'default' / 'go to' approach

→ improvements have been proposed but it's still pretty good!

let's look at
ingredients!

'Per-data-point' squared errors

So far we have focussed on 'least squares' type objectives, $\|y - X\theta\|^2$ or $\|y - f(x; \theta)\|^2$
↑ parameter to estimate.

These can be written as eg

$$\begin{aligned}\|y - X\theta\|^2 &= \sum_{i=1}^m (y_i - (X\theta)_i)^2 & \text{row } i \text{ of } X \equiv x_i^T \\ &= \sum_{i=1}^m (y_i - (\text{row}_i X) \theta)^2 = \sum_{i=1}^m (y_i - x_i^T \theta)^2\end{aligned}$$

ie sums of squared error over individual data points, per data point errors: $y_i - x_i^T \theta$

$$\Rightarrow \|y - X\theta\|^2 = \sum_{i=1}^m \text{'squared error for } i^{\text{th}} \text{ data point'}$$

why/where? →

Expected squared errors?

Minimising $\|y - X\theta\|^2 = \sum_{i=1}^m (y_i - x_i^T \theta)^2$ is the

same as minimising $\frac{1}{m} \sum_{i=1}^m (y_i - x_i^T \theta)^2$
extra factor

This can be thought of as an empirical /
sample-based estimate of the 'population'
expected value

$$E[(y - x^T \theta)^2] \approx \frac{1}{m} \sum_{i=1}^m (y_i - x_i^T \theta)^2$$

↑ expected value ↑ Random vars ↑ Sample realisations

Many decision/fitting problems can
be phrased in terms of minimising
expected 'loss' (760?, 2024...)

└→ loss? →

→ Hence we often get objectives that
can be written as sums over each
data point loss

Loss functions? Data misfit measure!

More generally we can define other 'data misfit'
functions that can be written as the expected
value of the 'per data point' losses/misfits.

→ 'empirical risk minimisation', 'M-estimation' etc

→ see e.g. 760 (2024...)

← max likelihood-like.

$$E[\ell(y, f(x; \theta))] \approx \frac{1}{m} \sum_{i=1}^m \ell(y_i, f(x_i; \theta))$$

random vars target (population) error empirical estimate / training error

$\ell(y_i, f(x_i; \theta))$ is the 'loss' (≥ 0)

for the ith data point given parameter θ .

We solve the 'empirical loss minimisation':

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \ell(y_i, f(x_i; \theta))$$

$$\text{Here we focus on } \ell(y_i, f(x_i; \theta)) = (y_i - x_i^T \theta)^2$$

$$\Leftrightarrow \sum_{i=1}^m \ell(y_i, f(x_i; \theta)) = \|y - X\theta\|^2$$

ie squared error loss

↳ others widely used!

Expected gradients

when we have a data misfit of the form:

$$| E[l(\gamma, f(x; \theta))] \approx \frac{1}{m} \sum_{i=1}^m l(\gamma_i, f(x_i; \theta)) |$$

It holds in general (modulo some conditions-->)

$$| \nabla_{\theta} E[l(\gamma, f(x; \theta))] = E[\nabla_{\theta} l(\gamma_i, f(x_i; \theta))] |$$

in the 'population' ('infinite dataset') &

$$| \nabla_{\theta} \frac{1}{m} \sum_{i=1}^m l(\gamma_i, f(x_i; \theta)) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} l(\gamma_i, f(x_i; \theta)) |$$

in the 'training' (observed) dataset

(i.e. the gradient of the total/expected loss
= the total/expectation of the gradients of the individual losses &

$$\text{Eg } \nabla_{\theta} [E[l(\gamma, f(x; \theta))] \approx \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} l(\gamma_i, f(x_i; \theta))$$

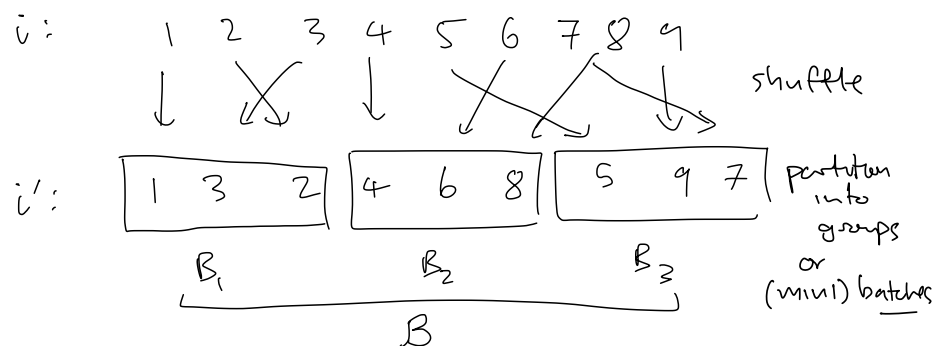
$\Rightarrow$ estimate of grad. = expected value of data point grads.

Decompositions

Additive loss functions & their gradients

$$\sum_{i=1}^m (\gamma_i - x_i^T \theta)^2 \quad \& \quad \sum_{i=1}^m \nabla_{\theta} (\gamma_i - x_i^T \theta)^2$$

can clearly be computed by summing in any order
&/or in 'groups' to be added together, e.g.



$$\sum_{i=1}^m (\gamma_i - x_i^T \theta)^2 = \sum_{B_j \in B} \sum_{i \in B_j} (\gamma_i - x_i^T \theta)^2$$

sum over sets of data points (groups) in partition B

sum over data points in group B_j

leads to iden of $\rightarrow$

Stochastic & minibatch gradient descent

The term 'stochastic gradient descent' is used for both a general family of methods & a special case of this family

More precisely we should call these

1. Standard or 'batch' gradient descent
2. Stochastic gradient descent
3. 'mini-batch' gradient descent

But we often call 2 & 3 'stochastic gradient descent' for short.

Idea

The key idea is simple: instead of solving

$$\left| \min_{\theta} \sum_{i=1}^m (y_i - f(x_i; \theta))^2 \right|$$

or computing

$$\left| \nabla_{\theta} \sum_{i=1}^m (y_i - f(x_i; \theta))^2 \right| \text{ etc.}$$

over the whole (training) dataset, we instead compute them over various subsets of the data called 'minibatches'

Note: when being precise we call

full dataset $\longleftrightarrow$ batch

subset of data $\longleftrightarrow$ minibatch

but again sometimes people say 'batch' e.g. 'batch size' instead of 'minibatch' (which is what they should say)

Gives:

- o batch gradient descent: full data
- o minibatch gradient descent: subsets of data
- o stochastic gradient descent: single data points

Using minibatches in gradient descent

Instead of updates like $\Theta_{k+1} \leftarrow \Theta_k - \alpha \nabla_{\Theta} \|y - A\Theta\|^2$ ^{for linear least sq.}

$$\Theta_{k+1} \leftarrow \Theta_k - \alpha \nabla_{\Theta} \|y - A\Theta\|^2$$

over full dataset

We do a series of updates over subsets (minibatches) of data, chosen randomly (hence 'stochastic')

eg an update looks like

$$\Theta_{k+1} \leftarrow \Theta_k - \alpha \nabla_{\Theta} \|y - A\Theta\|^2$$

over minibatch

→ can think of doing updates using different (noisy) estimates of corresponding expected loss / expected gradient

↳ some subtleties re scaling of gradient/update in terms of minibatch sizes, sample size, etc.

Minibatch gradient descent

- We will randomly divide training data into non-overlapping subsets called minibatches
- loop through minibatches, updating as we go based on minibatch gradient until all minibatches used
- That gives nmbatch = 'number minibatches' updates or iterations & one 'epoch' (all minibatches seen) so far.
- We then repeat: redivide training data, iterate through minibatches
- do this for n-epoch = 'number of epochs'
- gives n-mb × n-epoch total parameter updates (iterations)

Terms: iterations (parameter updates)
minibatches (data subsets in partition)
epochs (~~x~~ times seen whole dataset)

Algorithm: Mini-batch gradient descent
for $\|y - f(x; \theta)\|^2$ (nonlinear least sq.).

$\alpha \leftarrow$ learning rate (fixed here; can vary in general)

$\theta_0 \leftarrow$ initial parameter-estimate for iteration 1

For $i = 1 : n\text{-epoch}$

Randomly divide dataset into $n\text{-mbatch}$ minibatches

For $j = 1 : n\text{-mbatch}$

(iteration $k = (i-1) \cdot n\text{-mbatch} + j$)

$\theta_k^0 = \theta_{k-1}$ initial guess

Compute $g_k = \nabla_{\theta} \|y - f(x; \theta_k^0)\|^2$
for minibatch j
of epoch i

Update $\theta_k \leftarrow \theta_k^0 - \alpha g_k$

record loss $l_{ij} = l_k = \|y - f(x; \theta_k)\|^2$
for minibatch

Epoch loss(i) = $\sum_j l_{ij}$

Notes

- don't need to load whole dataset into memory
- actually improves 'generalisation'
i.e. regularisation towards good out-of-sample prediction models
- can analyse via estimators of expectations but...
- still not well understood!
- gradients computed for large neural networks via 'fancy chain rule'
 - ↳ pytorch & tensorflow etc implement derivative propagation for you!
- Many improvements, tricks etc possible!
- Estimate of average gradient: divide by minibatch size
- Estimate of total gradient: divide by minibatch size & multiply by total data size

Example : deconvolution revisited

Setup :

```
nepoch = 100
nmb = 50
xs = np.zeros((nepoch*nmb, len(x)))
xs0 = np.zeros(len(x))
Lx_p_norms = np.zeros(nepoch*nmb)
p = 1
data_squared_norms = np.zeros(nepoch*nmb)
data_squared_norms_matrix = np.zeros((nepoch, nmb))
epoch_exact_data_squared_norms = np.zeros(nepoch)
epoch_Lx_p_norms = np.zeros(nepoch)

# For norm
L = (np.diag(-np.ones(len(x))) + np.diag(np.ones(len(x)-1, 1))[:, :-1, :])

obs_indices = np.arange(0, len(y_noisy), dtype=int)
np.random.shuffle(obs_indices)
minibatch_partition = np.array_split(obs_indices, nmb)

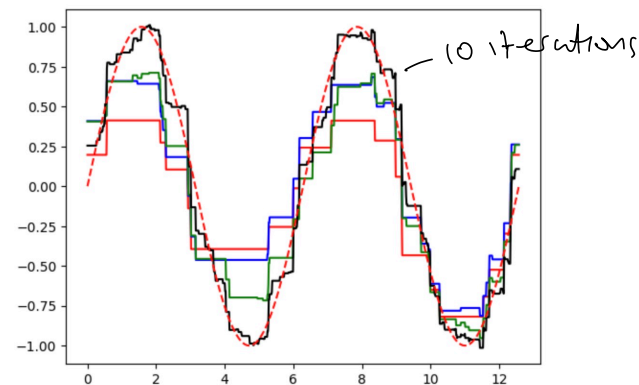
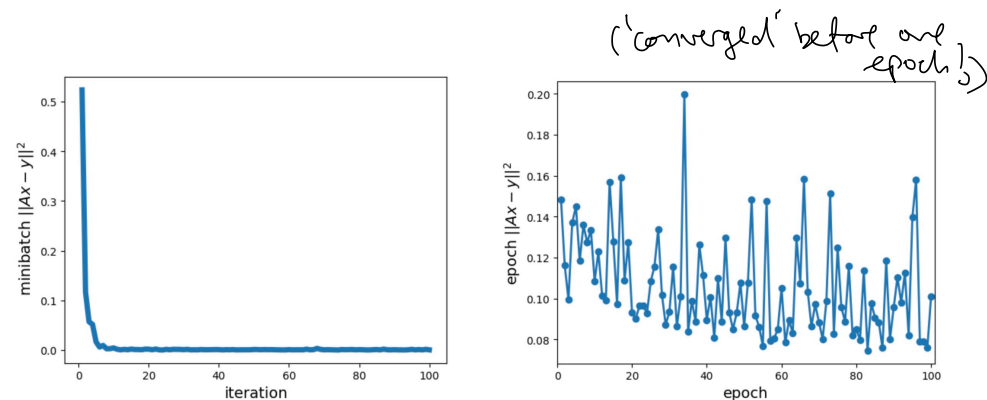
ind = minibatch_partition[0]
ind.sort()
A_mb = A[ind, :]

# get largest singular value of A for setting alpha
from scipy.sparse.linalg import svds
s1 = float(np.max(svds(A, k=1, return_singular_vectors=False)))
print(s1)
c = 1.0 # 0 < c < 2
alpha = c/(s1**2)
```

Loop :

```
# epoch loop
for i in range(0, nepoch):
    np.random.shuffle(obs_indices)
    minibatch_partition = np.array_split(obs_indices, nmb)
    # minibatch loop
    for j in range(0, nmb):
        mb_indices = minibatch_partition[j]
        mb_indices.sort()
        A_mb = A[mb_indices, :]
        k = i*nmb + j # global iteration index
        #update rule: scale by N/B to estimate total gradient
        N = len(y_noisy)
        B = len(mb_indices)
        xs[k, :] = xs0 + alpha*(N/B)*A_mb.T@(y_noisy[mb_indices] - A_mb@xs0)
        xs0 = xs[k, :]
        #calc norms
        Lx_p_norms[k] = np.linalg.norm(L@xs0, p)**p
        #minibatch norm
        data_squared_norms[k] = np.linalg.norm(y_noisy[mb_indices] - A_mb@xs0)**2
        data_squared_norms_matrix[i, j] = np.linalg.norm(y_noisy[mb_indices] - A_mb@xs0)**2
    epoch_exact_data_squared_norms[i] = np.linalg.norm(y_noisy - A@xs0)**2
    epoch_Lx_p_norms[i] = np.linalg.norm(L@xs0, p)**p
```

Example : 100 minibatches of size 10



Here: piecewise const? updates are
linear combo of subset
of A^T col (A rows)

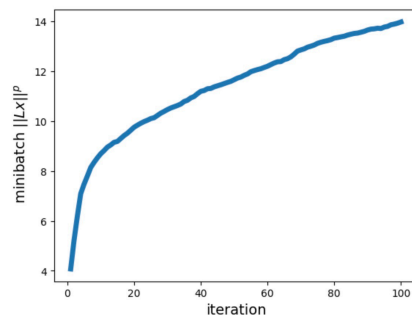
Tracking model complexity?

→ plot roughness measure

$\|D_1 x\|_1 \leftarrow$ total variation
(eg since took piecewise const.)

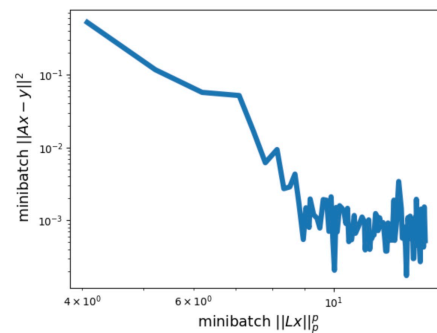
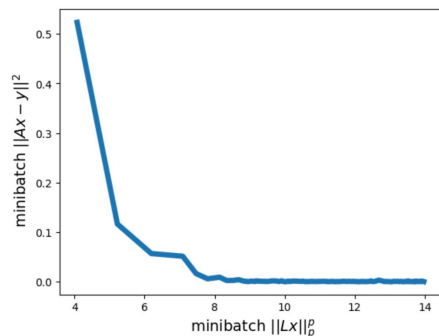
$$L = D_1$$

$$p = 1$$

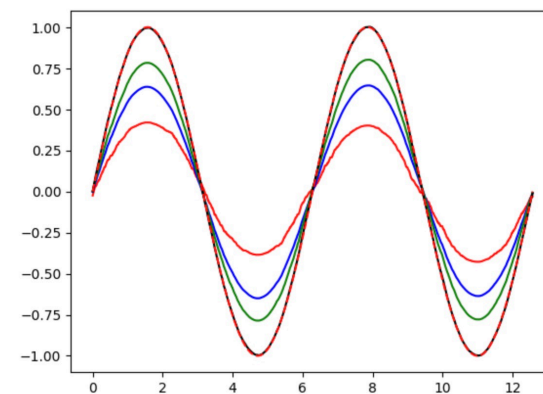
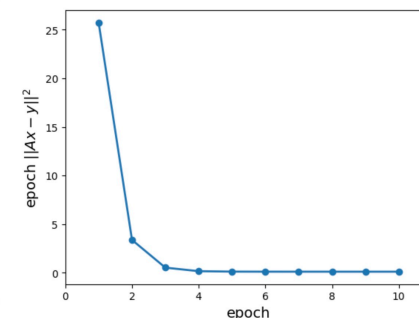
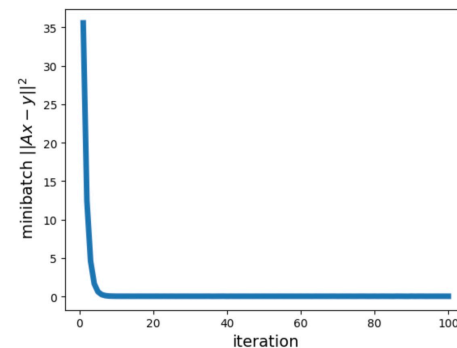


L-curves:

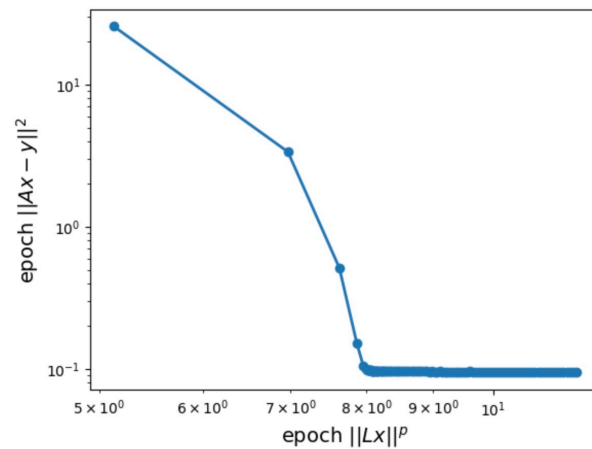
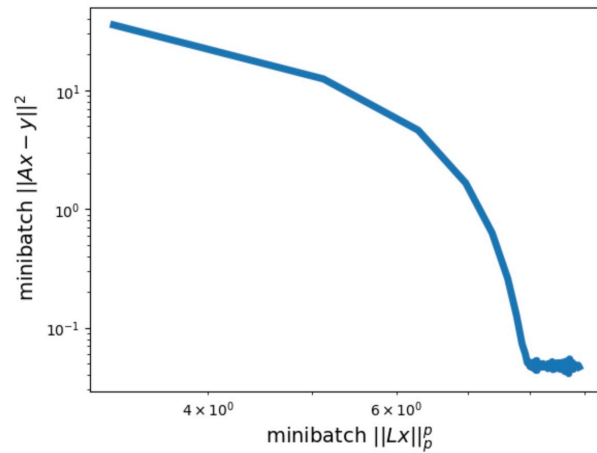
log scale



Example : 2 minibatches of size 500
→ back to Landweber /
full grad. descent
style.



Complexity



Exercise:

o Try on polynomial (x^{19})
example!

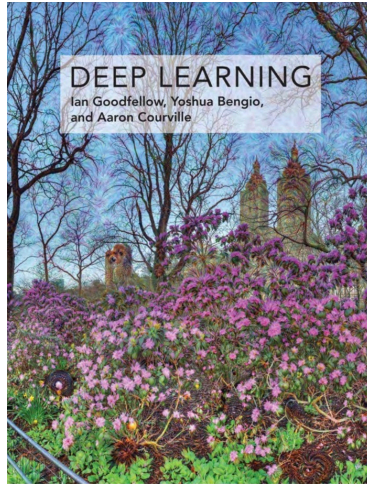
Tut:

- o Describe the minibatch version of gradient descent and contrast it with standard gradient descent. What is the minibatch size for a) stochastic gradient, and b) standard gradient descent. What is another name for standard gradient descent in machine learning terminology.
- o Suppose we carry out minibatch gradient descent for three epochs. How many times has the full training data set been used? If we use 20 minibatches, each having 10 samples, how many parameter updates (gradient descent iterations) have occurred after three epochs?

Further info & extensions

References:

(Goodfellow,
Bengio &
Courville)



Covers extensions eg
momentum

general models & losses

further alg.

+ much more!